Engineering an Improved Pine Screener

Whitepaper on architecture, factor selection, execution logic, and validation for TradingView Pine Script v6

Objective	Upgrade a 50D SMA screener into a higher-quality stock selection and execution framework.
Primary platform	TradingView Pine Screener and Pine Script v6.
Scope	Architecture, code-level improvements, strategy design, and validation roadmap.

Core thesis

The existing script is technically sound as a baseline filter, but the highest-value upgrade is to turn it into a layered system that separates setup qualification from entry timing and ranking. The screener should identify candidates; a separate strategy harness should validate entries, exits, and costs.

Prepared on 12 March 2026

Executive summary

The original Pine script correctly enforces daily data and uses barmerge.lookahead_off, which makes it a clean, non-repainting 50-day SMA state filter. However, it is still only a state filter. A robust stock-selection workflow in Pine Screener must be designed around the platform's rules: one indicator per screen, filtering through plot() and alertcondition() outputs, automatic exposure of the first ten plots, a maximum of five separate request.*() calls, and calculations over only the last 500 bars [1]. Within those constraints, the strongest improvement is to replace a single binary condition with a layered architecture that separates setup qualification, entry timing, and ranking.

This whitepaper proposes an enhanced design built around five ideas: (1) trend structure rather than price-vs-average state alone, (2) relative strength against a benchmark, (3) liquidity and volatility gates to improve tradability, (4) event-based entry logic for breakouts or pullbacks, and (5) optional fundamental overlays that fit within Pine's request budget [1]-[3]. The result is a screener that can rank candidates rather than merely mark them as above or below a 50-day moving average.

- Use one tuple-based request.security() call for the main daily dataset so multiple series consume only one request slot [2][3].
- Expose SetupPass, TradePass, and Score as separate outputs so screening, sorting, and alerting each have a clear role [1].
- Add relative strength, average dollar volume, ATR percent, and relative volume to distinguish quality breakouts from noisy ones [8]-[11].
- Convert state alerts into event triggers, and convert the indicator into a separate strategy harness for exit design and cost modeling [4]-[6].
- Maintain modular screener variants instead of one monolithic script when additional modules would exceed the five-request ceiling [1].

1. Baseline review of the current script

The current implementation has three notable strengths. First, it forces daily data so the 50-day moving average remains a true daily measure regardless of the chart timeframe. Second, it uses lookahead_off, which is the correct default for higher-timeframe requests and avoids leaking future information into historical bars [2]. Third, it already emits screener-friendly outputs: a hidden binary plot for filtering and a visible numeric plot for ranking by distance from the 50-day average [1].

Its limitations are strategic rather than syntactic. 'Above the 50D SMA' is useful as a coarse regime flag, but it does not differentiate between strong trends and weak drift, liquid names and thin names, or timely entries and stale moves. In practice, traders need a setup filter, a trigger, and a ranking mechanism, not only a single state variable.

Table 1. Original script review and recommended upgrades.

Baseline element	Assessment	Recommended augmentation
Daily data enforcement	Strong	Keep it. Preserve true daily semantics regardless of chart timeframe.
lookahead_off on higher-timeframe requests	Strong	Keep it as the default anti-bias posture [2].
Hidden binary plot and visible numeric plot	Strong	Expand to SetupPass, TradePass, and Score outputs [1].

Baseline element	Assessment	Recommended augmentation
Single above/below SMA state	Limited	Replace with layered trend, strength, liquidity, and trigger logic.
No tradability filter	Weak	Add average dollar volume, ATR percent, and relative volume.
State-based alert only	Weak	Trigger alerts only when a new trade state appears [6].
No exit or cost model	Missing	Backtest in a dedicated strategy() script with costs and Bar Magnifier [4][5].

2. Platform constraints that should drive the design

Pine Screener is not a general-purpose backtesting engine. It is a scanning interface whose rules should directly shape the codebase. The selected script must emit at least one plot() or alertcondition() output, only one indicator can be used per screen, the first ten plots are automatically added as table columns, and the screener supports no more than five separate request.*() calls in the scanned script [1]. These rules are not small implementation details; they determine what is practical to build.

Table 2. Pine Screener constraints and why they matter.

Constraint	Design implication
One indicator per screen [1]	All usable columns, filters, and alerts must live inside a single script.
First ten plots auto-added [1]	Choose the first ten plots intentionally; treat them as scarce user-interface slots.
Plots and alert conditions are the filter surface [1]	Binary flags and ranking numbers should be explicit outputs, not hidden internal variables only.
Max five separate request.*() calls [1]	Merge same-context requests, and split advanced features into separate screener variants when needed.
input.symbol(), input.timeframe(), and input.time() are ignored by the screener [1]	Use input.string() for benchmark symbols and make defaults sensible.
Only the last 500 bars are calculated [1]	Keep lookbacks compact and use a separate strategy script for deep-history evaluation.
Supported timeframes are limited [1]	Request only screener-supported strings such as 1D, 1W, or 1M.

Design consequence
A feature-rich script that ignores these constraints will either fail to scan or become hard to use. The best Pine Screener architecture is not the most feature-heavy script; it is the highest-signal script that respects plot slots, request budget, and lookback limits.

3. Proposed enhanced screener architecture

The recommended architecture is a layered pipeline: daily data -> setup filter -> entry trigger -> output columns -> execution and validation. The setup filter answers 'Is this symbol structurally attractive?' The trigger answers 'Is now a timely place to act?' The output layer answers 'How do I sort and operationalize the result?' This separation makes the script easier to scan, easier to test, and easier to extend.

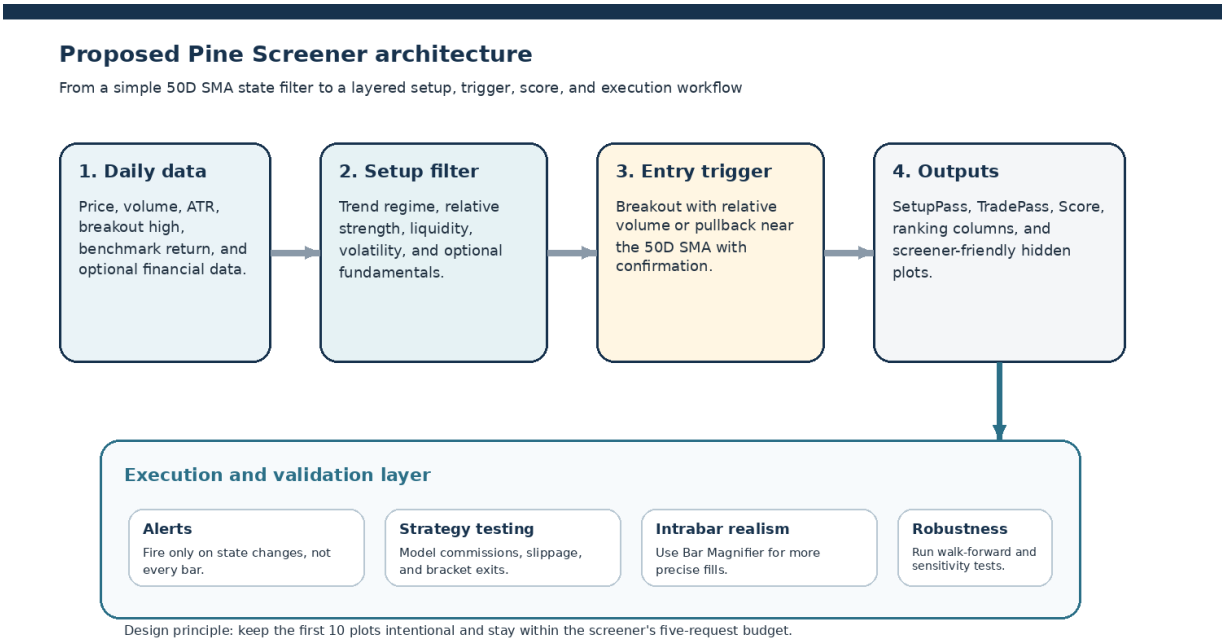

Figure 1. Proposed screening and trade-decision pipeline.

3.1 Trend regime filter

For long-side stock selection, a stronger trend definition is close > SMA50, SMA50 > SMA200, SMA50 > SMA50[slopeBars], and SMA200 > SMA200[slopeBars]. This turns the screener from a single-level check into a regime-aware filter. The broader trend-following and momentum literature provides the conceptual rationale for preferring structured trends over isolated price conditions [8]-[10].

3.2 Relative strength against a benchmark

A stock that is above its 50-day average but lagging the market is materially different from a stock that is outperforming the market over the same horizon. A practical screener metric is a 63-day stock return minus a 63-day benchmark return, where the benchmark can default to AMEX:SPY and be selected through input.string() because input.symbol() values are ignored by Pine Screener [1]. Relative-strength ranking is aligned with the long empirical record for momentum in both cross-sectional and time-series settings [9][10].

3.3 Liquidity and volatility gates

Tradability should be a first-class design variable. Add a minimum average daily dollar volume floor, a maximum ATR-as-percent-of-price cap, and a relative-volume condition for breakout entries. This reduces the number of thin, hard-to-execute names that pass the screen. The SEC's own discussion of thinly traded securities highlights wider transaction costs and difficulties for meaningful entry and exit in low-volume names [11].

3.4 Event-based entry logic

Setup quality and entry timing should not be conflated. The same strong stock can deserve a breakout entry in one phase and a pullback entry in another. A practical design is to keep both entry modes as options: a breakout above a prior 20-day high with relative volume, or a pullback that remains above the 50-day average and confirms by closing above the prior day's close. This turns the screener from a static watchlist sorter into a timing-aware tool.

3.5 Optional fundamental overlay

Pine can request issuer financial data with request.financial(), including metrics such as EPS growth, revenue growth, and direct financial identifiers. Because not every issuer publishes every period, the overlay should degrade gracefully: use ignore_invalid_symbol = true and treat missing data as non-blocking unless the strategy explicitly requires it [2]. The purpose of the overlay is not to force a deep fundamental model into Pine, but to bias the scan toward names where price strength is not completely disconnected from business momentum. Piotroski-style quality filters are a sensible reference point for this layer [12].

3.6 Composite score and modular variants

Expose at least three top-level outputs: SetupPass, TradePass, and Score. Score can be a weighted sum of trend, relative strength, liquidity, volatility, fundamentals, and trigger quality. The exact weights matter less than having a stable, interpretable ranking that can be tested. Importantly, the request ceiling matters: one symbol tuple request plus one benchmark request plus three financial requests already consumes the screener's five-request budget [1]. Advanced modules such as earnings-event filters or sector-ETF overlays should therefore live in separate script variants rather than a single overloaded script.

4. Pine v6 implementation patterns that matter

4.1 Compress same-context requests with tuples

TradingView's documentation explicitly recommends condensing several same-context requests into a single tuple-based request.*() call. A tuple request still counts as one request, not multiple, and can reduce runtime, memory usage, and compiled size [2][3]. For Pine Screener, this is not merely an optimization; it is the difference between fitting inside the five-request ceiling and exceeding it.

Representative pattern 1. A tuple request that keeps multiple daily series inside one request slot.

```
[dClose, sma50, sma200, atr14, avgVol50, prevBreakHigh, ret63] = request.security(
    syminfo.tickerid,
    "1D",
    [
        close,
        ta.sma(close, len50),
        ta.sma(close, len200),
        ta.atr(atrLen),
        ta.sma(volume, volLen),
        ta.highest(high, breakLen)[1],
        close / close[rsLen] - 1.0
    ],
    lookahead = barmerge.lookahead_off
)
```

4.2 Guard against invalid or sparse financial data

request.financial() can fail when the requested metric or period does not exist for a symbol. The ignore_invalid_symbol parameter lets the function return na instead of halting the script, which is exactly the behavior desired in a screener that must scan many names with uneven disclosure coverage [2].

4.3 Keep higher-timeframe requests non-misleading

The lookahead parameter controls whether higher-timeframe requests can leak future data into the historical past. The Pine docs warn that using lookahead incorrectly on higher-timeframe requests is misleading because it can make historical results appear better than what was knowable in real time [2]. The safest default is the one already present in the original script: lookahead = barmerge.lookahead_off.

4.4 Use the first ten plot slots intentionally

Because Pine Screener automatically adds the first ten plots as columns, these slots should be curated. Hidden binary plots are useful for machine-style filters, while visible numeric plots are better for ranking. A practical slot map is shown below.

Table 3. Recommended use of the first ten plot slots in Pine Screener.

Plot slot	Suggested output	Purpose
1	Setup Pass	Binary filter for structural qualification.
2	Trade Pass	Binary filter for actionable timing.
3	Score	Primary ranking metric.
4	RS vs Benchmark %	Relative-strength ranking.
5	% From 50D	Distance from the mean; useful for pullback management.
6	ATR %	Volatility control and stop calibration.
7	Avg $ Volume	Tradability filter.
8	Relative Volume	Breakout quality signal.
9	Piotroski F or quality metric	Optional fundamental overlay.
10	EPS 1Y Growth %	Optional growth confirmation.

4.5 Compile-safe formatting habits

Pine line wrapping is stricter than many users expect. The style guide notes that wrapped lines outside parentheses cannot use indentation that is a multiple of four, whereas wrapped expressions inside parentheses can use any indentation style [7]. In practice, the safest habit is to keep long function calls on the same line as the assignment token when practical and to wrap multiline Boolean expressions inside parentheses.

Representative pattern 2. Parenthesized multiline logic that avoids common continuation errors.

```
trendOk = (
    hasData and
    dClose > sma50 and
    sma50 > sma200 and
    sma50 > sma50Past and
    sma200 > sma200Past
)
```

5. Strategy augmentation beyond screening

The screener should nominate candidates; the strategy harness should validate whether the resulting trades are executable and attractive after costs. Pine documentation separates indicator alerts from strategy behavior for a reason. The scanning script should stay efficient and screener-friendly, while a dedicated strategy() version can model commissions, slippage, fill assumptions, and exit logic [4]-[6].

5.1 Event-based alerts, not state-based alerts

An alert that fires whenever aboveSma is true is a state monitor, not an entry signal. For execution, alerts should fire only when the trade state changes, for example when tradePass becomes true after being false on the prior bar. TradingView's alert documentation makes clear that alertcondition() defines triggers that users must then create through the alert dialog; the quality of the trigger condition itself therefore matters [6].

Representative pattern 3. Event-based alerting rather than repeated state alerts.

```
alertcondition(
    tradePass and not tradePass[1],
    title = "New Trade Pass",
    message = "Symbol just qualified for the screen"
)
```

5.2 Exit and risk design

The first strategy version should keep exits simple and explicit: an ATR-based initial stop, a bracket profit target, and an optional time stop for trades that fail to follow through. Pine's strategy.exit() supports explicit stop and limit levels, and the strategy FAQ describes the difference between price-based and tick-based exit parameters [5]. Once the baseline is validated, a trailing stop can be added after the trade reaches a predefined R multiple.

5.3 Realistic backtesting assumptions

Backtests that ignore intrabar execution and trading frictions tend to overstate edge. TradingView documents both slippage and commission in strategy settings, and Bar Magnifier can improve order-fill realism by using lower-timeframe data rather than only the broker emulator's default intrabar assumptions [4]. These controls belong in the strategy harness even if they do not belong in the screener.

5.4 Portfolio-level overlays

Once the signal itself is stable, portfolio construction becomes the next source of improvement. Limit the number of simultaneous positions, cap exposure to any one sector or theme, and avoid taking multiple highly correlated breakout names at the same time. These overlays are not usually handled inside Pine Screener itself, but they strongly influence realized results.

6. Validation framework

A key operational point follows directly from Pine Screener's 500-bar calculation limit: the screener is appropriate for current selection, not for deep historical research [1]. Historical validation should happen in a separate strategy script on the chart. The objective is not to maximize one in-sample equity curve, but to identify a stable screening architecture whose outputs remain useful across regimes and transaction-cost assumptions.

Table 4. Suggested validation checklist for the upgraded strategy.

Test	Why it matters	Practical decision rule
Baseline vs enhanced comparison	Shows whether the extra architecture adds value.	Keep the complexity only if expectancy, hit rate, or turnover-adjusted return improves net of costs.
Commission and slippage sensitivity	Many screens fail once realistic friction is added.	Evaluate multiple cost levels; reject a model that only works at zero-friction assumptions.
Walk-forward validation	Reduces the odds of choosing thresholds that only fit one sample.	Freeze parameters after each training window and score the next segment out of sample.
Regime segmentation	Signals often behave differently in strong vs weak tape.	Inspect results when the broad market is above vs below its long-term trend.
Parameter stability	Narrow peaks usually indicate overfitting.	Prefer wide plateaus where nearby settings behave similarly.
Universe hygiene	Results can be distorted by survivorship or unrealistic watchlists.	Use the actual watchlists you plan to scan, and document any selection bias.

Practical rule
Do not optimize the screener and the exit model simultaneously at the beginning. First stabilize candidate selection. Then test whether a small family of exit models improves outcomes without making the strategy fragile.

7. Implementation roadmap

Table 5. Suggested implementation sequence.

Phase	Primary deliverable	Notes
Phase 1	Core technical screener	Tuple request, SetupPass, TradePass, Score, RS, liquidity, ATR, and event-based alerts.
Phase 2	Strategy harness	Separate strategy() script with commission, slippage, Bar Magnifier, and basic ATR bracket exits.
Phase 3	Robustness study	Walk-forward, parameter sensitivity, and regime segmentation across the intended watchlists.
Phase 4	Variant library	Separate scripts for technical-only, technical-plus-fundamentals, or sector-relative-strength versions when modules exceed the five-request budget.

The most important governance decision is to resist turning every idea into a single script. Under Pine Screener's request and plot limits, a compact and interpretable family of scripts is often superior to one script that attempts to do everything.

Conclusion

Your original script is a solid foundation because it already respects daily data semantics and avoids obvious repainting issues. The next step is not cosmetic. It is architectural. A professional-grade Pine Screener should separate setup from trigger, rank candidates rather than only flag them, and reserve a separate strategy harness for exit logic and cost-aware validation. Within Pine Screener's constraints, the highest-leverage additions are trend structure, benchmark-relative strength, liquidity and volatility gates, event-based entry logic, and deliberate use of the first ten plot outputs [1]-[4].

If implemented carefully, the upgraded codebase will be more useful operationally in three ways: it will surface stronger candidates, it will sort those candidates more intelligently, and it will make downstream testing and execution less ambiguous. That is the difference between a chart condition and a screening framework.

Appendix A. Representative Pine v6 building blocks

The snippets below are not meant to be copied verbatim as a complete strategy. Their purpose is to show the main building blocks in a compile-safe structure.

Appendix snippet A1. Daily tuple request.

```
// Main daily tuple request
[dClose, dPrevClose, dVol, sma50, sma200, sma50Past, sma200Past,
 atr14, avgVol50, prevBreakHigh, ret63] = request.security(
    syminfo.tickerid,
    "1D",
    [
        close,
        close[1],
        volume,
        ta.sma(close, len50),
        ta.sma(close, len200),
        ta.sma(close, len50)[slopeBars],
        ta.sma(close, len200)[slopeBars],
        ta.atr(atrLen),
        ta.sma(volume, volLen),
        ta.highest(high, breakLen)[1],
        close / close[rsLen] - 1.0
    ],
    lookahead = barmerge.lookahead_off
)
```

Appendix snippet A2. Setup vs trigger separation.

```
trendOk = (
    hasData and
    dClose > sma50 and
    sma50 > sma200 and
    sma50 > sma50Past and
    sma200 > sma200Past
)

setupPass = trendOk and liquidityOk and volatilityOk and rsOk
tradePass = setupPass and entryOk
```

Appendix snippet A3. Explicit bracket exit structure.

```
if longSignal and strategy.position_size == 0
    strategy.entry("L", strategy.long)

stopPrice  = strategy.position_size > 0 ? strategy.position_avg_price - 2.0 * atr14 : na
limitPrice = strategy.position_size > 0 ? strategy.position_avg_price + 3.0 * atr14 : na

strategy.exit("L-X", "L", stop = stopPrice, limit = limitPrice)
```

References

[1] TradingView. "TradingView Pine Screener: key features and requirements." Help Center. Accessed March 12, 2026. Source link

[2] TradingView. "Other timeframes and data." Pine Script User Manual. Accessed March 12, 2026. Source link

[3] TradingView. "Profiling and optimization." Pine Script User Manual. Accessed March 12, 2026. Source link

[4] TradingView. "Strategies." Pine Script User Manual. Accessed March 12, 2026. Source link

[5] TradingView. "Strategies FAQ." Pine Script FAQ. Accessed March 12, 2026. Source link

[6] TradingView. "Alerts." Pine Script FAQ. Accessed March 12, 2026. Source link

[7] TradingView. "Style guide." Pine Script User Manual. Accessed March 12, 2026. Source link

[8] Hurst, Brian K., Yao Hua Ooi, and Lasse H. Pedersen. "A Century of Evidence on Trend-Following Investing." Journal of Portfolio Management, 2017. Source link

[9] Asness, Cliff, Andrea Frazzini, Ronen Israel, and Tobias J. Moskowitz. "Fact, Fiction and Momentum Investing." Journal of Portfolio Management, 2014. Source link

[10] Moskowitz, Tobias J., Yao Hua Ooi, and Lasse H. Pedersen. "Time Series Momentum." Journal of Financial Economics 104, no. 2 (2012): 228-250. Source link

[11] U.S. Securities and Exchange Commission. "SEC Issues Statement on Market Structure Innovation for Thinly Traded Securities." October 17, 2019. Source link

[12] Piotroski, J.D. "Value investing: The use of historical financial statement information to separate winners from losers." Journal of Accounting Research 38 (2000): 1-41. Source link